Course Outline

Sem		Course Name	Th	Tut	Pra	Hrs	Cr
Sem I	HSC	English Communication	2	0	0	2	2
	BSC	Mathematics – 1	3	1	0	4	4
	BSC	Applied Physics	3	0	0	3	3
	BSC	Applied Physics Laboratory	0	0	2	2	1
	BSC	Environment Science	2	0	0	2	2
	ESC	Workshop Practices	0	0	2	2	1
	BSC	Engineering Graphics	0	0	4	4	2
	ESC	Computer Programming -I / Elementary Programming –I	1	0	0	1	1
	ESC	Computer Programming -I Laboratory / Elementary Programming -I Laboratory	0	0	2	2	1
	HSC	Universal Human Values	1	0	0	1	1
	HSC	Indian Knowledge System	2	0	0	2	2
			14	**1**	**10**	**25**	**20**
Sem II	HSC	Professional Communication	2	0	0	2	2
	BSC	Mathematics – II	3	1	0	4	4
	BSC	Engineering Chemistry	3	0	0	3	3
	BSC	Engineering Chemistry Laboratory	0	0	2	2	1
	ESC	Elements of Electrical & Electronics Engineering	3	0	0	3	3
	ESC	Elements of Electrical & Electronics Engineering Laboratory	0	0	2	2	1
	ESC	Biological Systems for Engineers	2	0	0	2	2
	HSC	Yoga, Health & Hygiene OR National Cadet Corps (NCC) OR National Service Scheme (NSS)	0	0	2	2	1
	HSC	Organizational Behavior	1	0	0	1	1
	ESC	Computer Programming -II/ Elementary Programming – I	1	0	0	1	1
	ESC	Computer Programming -II Laboratory/ Elementary Programming – II Laboratory	0	0	2	2	1
			15	**1**	**8**	**24**	**20**
Sem III	Pro	Civic and Social Service Internship	0	0	0	0	1
	BSC	Discrete Mathematics	3	1	0	4	4
	PC	Database Management System	3	0	0	3	3
	PC	Database Management System Laboratory	0	0	2	2	1
	PC	Digital Logic and Design	3	1	0	4	4
	PC	Data Structures	3	0	0	3	3
	PC	Data Structures Laboratory	0	0	2	2	1
	PC	Object Oriented Programming	3	0	0	3	3
	PC	Object Oriented Programming Laboratory	0	0	2	2	1
			15	**2**	**6**	**23**	**21**

3rd Semester

24MA202T					Discrete Mathematical Structures (For CS, ICT)					
Teaching Scheme					**Examination Scheme**					
L	**T**	**P**	**C**	**Hrs./Week**	**Theory**			**Practical**		**Total Marks**
					MS	**ES**	**IA**	**LW**	**LE/Viva**	
3	**1**	**0**	**4**	**4**	**25**	**50**	**25**	**--**	**--**	**100**

COURSE OBJECTIVES

- To understand the concept of sets, relations, functions and logic.
- To understand Combinatorics as an analytical method for problem solving.
- To apply graph theory-based modeling to solve real life problems.
- To explain the basics of algebraic structures.

UNIT I: SETS, RELATIONS, FUNCTIONS AND LOGIC Finite and Infinite sets, Counting Principle, Principle of inclusion and exclusion, Mathematical Induction, Relations, Types of Relation, Equivalence relation, Partial Ordered Relations. Functions, types of functions, Logic and Propositional calculus – Simple and Compound Propositions, Basic logical operators. Truth tables. Tautologies, Contradictions, logical equivalence, conditional and bi-conditional statements. Valid arguments and Fallacy. Hasse diagram and Lattice.	**10 Hrs.**
UNIT II: COMBINATORICS Permutation, Combination, Pigeonhole principle, Extended Pigeonhole principle, Recurrence relations, Linear recurrence relations with constant coefficients, Solutions of recurrence relations: second order homogeneous linear recurrence relation, general homogeneous linear recurrence relation.	**10 Hrs.**
UNIT III: GRAPH THEORY AND ITS APPLICATIONS Graphs and related definitions, Sub graphs, Homomorphism and Isomorphism, Paths and Connectivity. Traversable and Eulerian graphs, and Konigsberg Bridge problem, Hamiltonian graphs, Labeled and weighted graphs. Complete, regular and bipartite graphs, Planar graphs, Graph coloring. Four color problem. Directed graphs, Strongly and weakly connected graphs, Trees, rooted trees, sequential representation of directed graphs, Adjacency matrix, Powers of the adjacency matrix.	**12 Hrs.**
UNIT IV: ALGEBRAIC STRUCTURES Group, Semi group, Monoids, Properties of a Group, Composition table for finite Group, Order of a group, Order of its elements, Cyclic Group, Generator, Lagrange's Theorem. Ring, Properties of Rings, Integral Domain, Field.	**10 Hrs.**
	42 Hrs.

COURSE OUTCOMES

On completion of the course, student will be able to:

CO1 - Remember the basics of counting and combinatorics.
CO2 - Understand the basic concepts of sets, relations, functions, logic and be able to determine their properties.
CO3 - Model and solve the real life problem using recurrence relations.
CO4 - Apply Graph theory in related areas such as shortest path problems and network flow problems.
CO5 - Defend and point out fallacious reasoning and propositions in algebraic structures.
CO6 - Model and solve any given engineering problem involving graphs and trees.

TEXT/REFERENCE BOOKS

1. Seymour Lipschutz and Marc Lipson, "Discrete Mathematics, Schaum's Series", McGraw-Hill Education.
2. Kenneth Rosen, "Discrete Mathematics and Its Applications", McGraw Hill Education.
3. Bernard Kolman, Robert Busyb and Sharon C. Ross, "Discrete Mathematical Structures", Pearson.
4. Thomas Koshy, "Discrete Mathematics with Applications", Academic Press Inc.
5. Ralph P. Gramaldi, "Discrete and Combinatorial Mathematics", Pearson.
6. C.L. Liu, D.P. Mohapatra, "Elements of Discrete Mathematics: A Computer Oriented Approach", McGraw Hill Education.

24CS202T					Database Management Systems					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
3	0	0	3	3	25	50	25	-	-	100

COURSE OBJECTIVES

- To learn fundamental concepts of Database management system
- To study various Database design models and normalization concepts
- To apply the above concepts to optimal Database design for various applications
- To carry out data retrieval and manipulation using SQL.

UNIT 1: Introduction And Database Models File Structure: Concepts of fields, records and files, Conventional file system, Advantages of Database management system, Applications, Data abstraction, Data independence, Three levels of the architecture, Database Users. Entity-Relationship Model, Relational Data Model, Relational algebra operators and syntax.	**10 Hrs.**
UNIT 2: SQL Basics of SQL, DDL, DML, DCL, Primary key, foreign key, unique, not null, check, IN operator, Functions, set operations, sub queries, correlated sub-queries, Use of group by, having, order by, join and its types, Exist, Any, All, view and its type.	**10 Hrs.**
UNIT 3: Normalization And File Organization Importance of a good schema design, Problems encountered with bad schema designs, dependency theory - functional dependencies, Armstrong's axioms for FD's, Minimal covers, 1NF, 2NF, 3NF and BCNF. File Organization: Sequential, Indexed and Random File Organization, indexing structure for index files, hashing for direct files, multi-key file organization and access methods, Storage strategies: Indices, B and B+ trees, hashing.	**10 Hrs.**
UNIT 4: Transactions, Query Processing, Security ACID properties, Schedules, Serializability, Concurrency control protocols, measures of query cost, selection operation, sorting, join. Security: Discretionary and Mandatory Access Control; Audit Trails; Multi-Level Security; Statistical Databases; Data Encryption. Introduction to NOSQL Databases.	**12 Hrs.**
	42 Hrs.

COURSE OUTCOMES

On completion of the course, student will be able to

CO1 - Understand the need of database management systems.
CO2 - Explain entity relationship and relational database models.
CO3 - Apply SQL commands in database systems.
CO4 - Analyze normalization and file organization techniques in database systems.
CO5 - Analyze concurrency control in database systems.
CO6 - Determine security levels in database systems.

TEXT/REFERENCE BOOKS

1. A Silberschatz, H F Korth and S Sudarshan, “Database System Concepts”, McGRAW Hill.
2. C. J. Date, A. Kennan, and S. Swamynathan, “An Introduction to Database Systems”, PersonEducation
3. RamezElmasri and Shamkant B Navathe, “Fundamentals of Database Systems”, Addison Wesley

24CS202P					Database Management Systems Laboratory					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
0	0	2	1	2	-	-	-	50	50	100

COURSE OBJECTIVES

- To learn fundamental concepts of Database management system
- To study various Database design models and normalization concepts
- To apply the above concepts to optimal Database design for various applications
- To carry out data retrieval and manipulation using SQL.

LIST OF EXPERIMENTS

1. **Database models:**
   Entity Relationship model, Relational model.
2. **Normalization Techniques:**
   1NF, 2NF, 3NF, BCNF
3. **DDL Commands in SQL:**
   Database and table creation, alteration, defining constraints, primary key, foreign key, unique, not null, check.
4. **DML Commands in SQL:**
   IN operator, inbuilt SQL functions - aggregate functions, built-in functions numeric, date, string functions, set operations, sub-queries, correlated sub-queries in SQL, group by, having, order by, join operations, exist, any, all, and types of views.
5. **DCL Commands in SQL:**
   Transaction control commands, Commit, Rollback, Save point.
6. **Implementation of Embedded SQL, PL SQL Concepts.**
7. **Implementation of Cursors, Stored Procedures, Stored Function, Triggers.**

COURSE OUTCOMES

On completion of the course, student will be able to

CO1 - Design Entity Relationship and Relational database models.
CO2 - Apply normalization techniques to database models.
CO3 - Apply SQL commands in database systems.
CO4 - Apply transaction control commands in database systems.
CO5 - Apply embedded and PL SQL commands in database systems.
CO6 - Create database systems for real time problems.

TEXT/REFERENCE BOOKS

1. Ivan Bayross, SQL, PL/SQL: "The Programming Language of Oracle", BPB Publications.
2. A Silberschatz, H F Korth and S Sudarshan, "Database System Concepts", McGRAW Hill.
3. C. J. Date, A. Kennan, and S. Swamynathan, "An Introduction to Database Systems", Person Education.

24CS201T					Digital Logic and Design					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
3	1	0	4	4	25	50	25	-	-	100

PREREQUISITES: Elements of Electrical & Electronics Engineering

COURSE OBJECTIVES

- To learn digital circuit design principles and basic concepts.
- To learn design of combinational circuits as per the application needs.
- To learn design of sequential circuits as per the application needs.

UNIT 1: Binary arithmetic and codes Introduction: Digital Systems; Data representation and coding; Logic circuits; integrated circuits; Analysis; design and implementation of digital systems. Truth table; Basic logic operation and logic gates. Number Systems and Codes: Positional number system; Binary; octal and hexadecimal number systems; Methods of base conversions; Binary; octal and hexadecimal arithmetic; Representation of signed numbers; Fixed and IEEE floating point numbers; Binary coded decimal codes; Gray codes; Error detection and correction codes - parity check codes and Hamming codes.	**8 Hrs.**
UNIT 2: Boolean Algebra and combinational circuits Boolean Algebra & Simplification of Boolean Algebra: Basic postulates and fundamental theorems of Boolean algebra; Standard representation of logic functions; The Map Method; SOP and POS forms; Simplification of switching functions K-map and Quine-McCluskey tabular methods; Synthesis of combinational logic circuits. Combinational Logic Modules and their applications: Decoders; encoders; multiplexers; demultiplexers and their applications; Parity circuits and comparators; Arithmetic modules- adders; sub tractors and ALU; Design examples.	**13 Hrs.**
UNIT 3: Sequential circuits and K-maps Sequential Logic systems: Definition of state machines; state machine as a sequential controller; Basic sequential circuits- latches and flip-flops: SR-latch; D-latch; D flip-flop; JK flip-flop; T flip-flop; Timing hazards and races; Analysis of state machines using D flip-flops and JK flip-flops; Design of state machines - state table; state assignment; transition/excitation table; excitation maps and equations; logic realization; Design examples. Finite State machine design and applications: Designing state machine using ASM charts; Designing state machine using state diagram; Design approaches for Synchronous and asynchronous machines; Registers and Counters; Application examples.	**14 Hrs.**
UNIT 4: Programmable Logic Devices **Logic Families:** Transistor-Transistor Logic (TTL); MOSFET; CMOS. **Programmable Logic Devices:** PLAs and their applications; Sequential PLDs and their applications; State-machine design with sequential PLDs; Introduction to field programmable gate arrays (FPGAs).	**7 Hrs.**
	42 Hrs.

TUTORIALS

Unit 1:	• Case Studies to design different circuits from basic gates. • simplify the circuits using boolean algebra. • Case studies and examples of binary codes and conversion.	**3 Hrs.**
Unit 2:	• Solve problems using K-map for simplification of boolean expression. • Examples to design and solve Encoder, Decoder, MUX, DEMUX, Adder Subtractor.	**5 Hrs.**
Unit 3:	• Real-world examples related to Flip Flops and design of counters. • Design of state machines - state table; state assignment; transition/excitation table; excitation maps and equations; logic realization	**5 Hrs.**
Unit 4:	• Case studies based on PLA, PLD, FPGA	**1 Hrs.**

COURSE OUTCOMES

On completion of the course, student will be able to

CO1- Understand the basics of Number Systems, Boolean algebra and Gates.
CO2- Design combinational circuits using logic gates.
CO3- Construct combinational circuits using multiplexers, demultiplexers, encoders and decoders.
CO4- Evaluate different types of logic families for digital circuit design.
CO5- Design and analyze sequential circuits.
CO6- Design digital logic circuits

TEXT/REFERENCE BOOKS

1. M Morris Mano, “Digital Logic and Computer Design”, Prentice Hall Publication
2. A. Anand Kumar, “Fundamentals of Digital Circuits”, PHI Learning Pvt. Ltd.
3. Malvino and Leach, “Principle of digital Electronics”, McGraw-Hill Education.

4. R.P. Jain, “Modern Digital Electronics”, McGraw-Hill.

24CS203T					Data Structures					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
3	0	0	3	3	25	50	25	--	--	100

COURSE OBJECTIVES

- To learn fundamental concepts of data and file structures.
- To implement various data structures and algorithms.
- To understand function of linear and non-linear data structures.
- To use suitable data structure in variety of applications.

UNIT 1: Introduction Datatypes: primitive vs non-primitive; Data structures: linear vs non-linear; Various Operations performed on DS; Performance Analysis and Measurement: Time and space analysis of algorithms; Asymptotic Notations, Searching and sorting in arrays	**08 Hrs.**
UNIT 2: Linear Data Structures **Stack**: Definition and concepts, Push and Pop operations; Representation in memory; Applications: Infix to prefix and postfix, evaluate postfix, recursion; **Queue**: Definition and concept, Enqueue and Dequeue operations; Representation in memory; Types: circular & priority queue. **Linked list**: Definition and concept, Insert and delete operations, Representation in memory, implementing stack and queue using LL, Types: Singly, Doubly, Circular.	**12 Hrs.**
UNIT 3: Non-Linear Data Structures **Trees**: Definition and concept; Memory representation; Types of trees: Binary, complete binary, full, perfect; Traversal: pre-order, in-order, post-order; Binary Search Trees; AVL trees; Min-Max Heap tree (with insert and delete operations for all trees). **Graphs**: Definition and concept; Memory representation; Types of graphs: complete, connected, directed/ undirected, weighted/ unweighted, cyclic/ non-cyclic; Traversal: breadth and depth first search.	**14 Hrs.**
UNIT 4: Hashing The symbol table, Hashing Functions: Division, Multiplication, Mid Square and Folding; Collision Resolution Techniques: Separate Chaining, Linear Probing, Quadratic Probing and Double Hashing.	**08 Hrs.**
	42 Hrs.

COURSE OUTCOMES

On completion of the course, students will be able to

CO1 - Recall the differences between primitive and non-primitive datatypes.
CO2 - Understand the concepts of linear and non-linear data structures.
CO3 - Apply search, insert and delete operation on data structures.
CO4 - Analyse the time and space complexity of algorithms.
CO5 - Evaluate trade-offs between linear and non-linear data structures.
CO6 - Develop strategies for implementing hash tables and resolving collisions.

TEXT/REFERENCE BOOKS

1. Tanenbaum, “Data Structures using C & C++”, Prentice-Hall International
2. Jean-Paul Tremblay & Paul G. Sorenson, “An Introduction to Data Structures with Applications”, Tata McGraw Hill
3. Sartaj Sahani, “Fundamentals of Data Structures in C++”, Galgotia.Publishers

24CS203P					Data Structures Laboratory					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
0	0	2	1	2	--	--	--	50	50	100

PREREQUISITES: Computer Programming - I

COURSE OBJECTIVES

- To learn fundamental concepts of Data and File Structures.
- To implement various data Structures and Algorithms.
- To understand function of linear and non-linear data structures.
- To use suitable data structure in variety of applications.

LIST OF EXPERIMENTS:

Arrays, Pointers and structures:

- Implementation of searching and sorting algorithms on arrays.
- Implementation of pointers and structures.
- Implementation of various operations using strings.

Linear Data structures:

- Implementation of Stack data structure and its applications.
- Implementation of various types of Queue data structure and their applications.
- Implementation of various types of Linked list data structure and their applications.

Non-linear Data structures:

- Implementation of binary tree and its traversals.
- Implementation of Binary search tree.
- Implementation of balanced trees.
- Implementation of various graph traversals: DFS & BFS.

Hashing:

- Implementation of Hash functions and tables.
- Implementation of collision resolution techniques.

COURSE OUTCOMES

On completion of the course, students will be able to

CO1 - Recall the differences between primitive and non-primitive datatypes.
CO2 - Apply searching and sorting algorithms on linear data structures.
CO3 - Implement search, insert, and delete operations on data structures.
CO4 - Implement hash tables and collision resolution techniques.
CO5 - Evaluate the complexity of algorithms.
CO6 - Design application using linear and non-linear data structures.

TEXT/REFERENCE BOOKS

1. Tanenbaum, “Data Structures using C & C++”, Prentice-Hall International
2. Jean-Paul Tremblay & Paul G. Sorenson, “An Introduction to Data Structures with Applications”, Tata McGraw Hill
3. Sartaj Sahani, “Fundamentals of Data Structures in C++”, Galgotia.Publishers

24CS204T					Object Oriented Programming Paradigms					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
3	0	0	3	3	25	50	25	-	-	100

COURSE OBJECTIVES

- Appraise the appropriate use of OOP Concepts in program design.
- Get acquainted with advanced OOP concepts including streams and multifile programs, exceptions, standard template library.
- Understand the use of performance optimization techniques.
- Understand various design patterns and best practices of Object-Oriented Software Design.

UNIT 1: Introduction to Object-Oriented Programming Overview of Object-Oriented Programming (OOP) principles, Difference between OOP and procedure oriented programming, Classes,, class members and Objects, Encapsulation, Instantiation, Data Hiding, Access Specifiers and their role in encapsulation, Constructors, Destructors and memory management through destructors, Execution order.	**12 Hrs.**
UNIT 2: Inheritance and Polymorphism Introduction to inheritance and polymorphism, Types of inheritance: single, multiple, multilevel, hierarchical, base class and derived class, Hierarchies and Multiple Inheritance, Function Overloading and Operator Overloading, Function overriding, Virtual Functions and Abstract Classes, Friend Functions, Static Function	**12 Hrs.**
UNIT 3: Advanced Concepts using C++ 'this' pointer, Dynamic Memory allocation, Streams and multifile programs, Templets and Exception handling in C++, Standard Template Library (STL) Overview and Usage, Performance optimization techniques.	**10 Hrs.**
UNIT 4: Design Patterns and Best Practices Design Patterns Overview: Creational, Structural, and Behavioural Patterns, Applying Design Patterns in C++, Object-Oriented Design Principles (SOLID), Code Refactoring Techniques, Unit Testing and Test-Driven Development (TDD)	**10 Hrs.**
	42 Hrs.

COURSE OUTCOMES

On completion of the course, student will be able to

CO1 - Understand object-oriented programming principles.
CO2 - Identify different programming constructs for a problem.
CO3 - Illustrate the use of object-oriented programming constructs.
CO4 - Compose exception handling framework.
CO5 - Select appropriate performance optimization techniques.
CO6 - Appraise design patterns and associated best practices.

TEXT/REFERENCE BOOKS

1. Object Oriented Programming in C, Robert Lafore
2. "The C++ Programming Language" by Bjarne Stroustrup
3. Y.P. Kanetkar ,"Let us C++", BPB Publication
4. "Design Patterns: Elements of Reusable Object-Oriented Software" by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides
5. "Programming: Principles and Practice Using C++" by Bjarne Stroustrup
6. "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo

24CS204P					Object Oriented Programming Paradigms Laboratory					
Teaching Scheme					Examination Scheme					
L	T	P	C	Hrs/Week	Theory			Practical		Total Marks
					MS	ES	IA	LW	LE/Viva	
0	0	2	1	2	-	-	-	50	50	100

COURSE OBJECTIVES

- Hands-on Object-Oriented Programming (OOP)
- Apply advanced OOP concepts including streams and multifile programs, exceptions, standard template library.
- Apply appropriate performance optimization techniques.
- Apply design patterns and best practices of Object-Oriented Software Design.

LIST OF EXPERIMENTS:

1. **Introduction to Object-Oriented Programming**:
   Implement a C++ program with a class representing a simple real-world entity (e.g., Student, Car, Employee). Define class members (attributes and methods) to represent the characteristics and behavior of the entity.
2. **Apply concept of encapsulations:**
   Utilize access specifiers to demonstrate encapsulation and data hiding.
3. **Demonstrate constructors and destructors:**
   Implement constructors to initialize class members and demonstrate the concept of instantiation by creating multiple objects of the class.
   Implement destructors to manage memory resources (e.g., dynamically allocated memory)
4. **Inheritance and its types:**
   Implementation of program to demonstrate inheritance. Types of inheritance: single, multiple, multilevel, hierarchical, base class and derived class, Hierarchies and Multiple Inheritance.
5. **Polymorphism and its types:**
   Implementation of program to demonstrate polymorphism. Types of polymorphism: compile-time polymorphism, run-time polymorphism.
6. **Implement overloading and overriding concepts:**
   Understand and demonstrate through implementation Function Overloading and Operator Overloading, Function overriding
7. **Abstraction and different types of functions in OOP:**
   Illustrate program executing Abstract Classes, Virtual Functions, Friend Functions, Static Function
8. **Pointer, Streams, Templets and multifile programs:**
   Execute the use of This pointer, Streams and multifile programs, Templets
9. **Exception handling:**
   handling of exceptions using the try and except statement.
10. **STL and optimization techniques:**
    Standard Template Library (STL) Overview and Usage and Introduction to performance optimization techniques.
11. **Unit Testing and Test-Driven Development (TDD)**
    Understanding Unit Testing and Test-Driven Development (TDD) and execute basic demonstration programs.

COURSE OUTCOMES

On completion of the course, student will be able to
CO1 - Apply object-oriented programming principles.
CO2 - Apply appropriate programming constructs for a problem.
CO3 - Illustrate the use of inheritance and polymorphism.
CO4 - Compose exception handling framework.
CO5 - Create a project based on OOP Concept.
CO6 - Demonstrate performance optimization, design patterns and associated best practices through a project.

TEXT/REFERENCE BOOKS

1. Object Oriented Programming in C, Robert Lafore
2. "The C++ Programming Language" by Bjarne Stroustrup
3. Y.P. Kanetkar ,"Let us C++", BPB Publication
4. "Design Patterns: Elements of Reusable Object-Oriented Software" by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides
5. "Programming: Principles and Practice Using C++" by Bjarne Stroustrup
6. "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo